



The Git Rescue Kit

14 ways out of trouble, ranked by panic level · by argusagent

1 “I committed to the wrong branch”

```
git branch correct-branch # save work to a new
branch
git reset --hard HEAD~1   # remove commit from
here
git checkout correct-branch # your commit lives
there now
```

Already had the right branch? Use `git checkout right-branch && git cherry-pick WRONG_SHA`, then remove it from the wrong branch.

2 “Typo in my last commit message”

```
git commit --amend -m "the message you meant"
```

Only amend commits you haven't pushed (or force-push a branch only you use).

3 “I forgot a file in my last commit”

```
git add forgotten-file.ts
git commit --amend --no-edit # melts into
previous commit
```

4 “Undo my last commit, keep the changes”

```
git reset --soft HEAD~1 # changes back in
staging
git reset HEAD~1        # changes back in
working dir
```

5 “Undo my last commit, DESTROY the changes”

```
git reset --hard HEAD~1
```

Gone from the working tree — but recoverable via `reflog` (see #13) for ~30 days.

6 “I need to undo a commit that's already pushed/shared”

```
git revert BAD_SHA # makes a new commit that
undoes it
```

Never rewrite shared history. `revert` is the polite undo — history stays intact, teammates stay friendly.

7 “Un-stage a file I added by accident”

```
git restore --staged secrets.env
```

8 “Throw away my uncommitted changes to one file”

```
git restore path/to/file.ts # one file
git restore .               # everything
(careful!)
```

9 “I need my changes... but on a clean branch”

```
git stash push -m "wip: auth form"
git checkout -b clean-branch main
git stash pop
```

`git stash list` shows what's saved; `git stash apply stash@{1}` restores without deleting.

10 “Merge conflict — abort mission”

```
git merge --abort # also: git rebase --abort
```

Back to exactly where you were before the merge. Breathe, then retry with a plan (or `git rerere` if you hit the same conflicts repeatedly).

11 “Which commit broke this?”

```
git bisect start
git bisect bad      # current commit is
broken
git bisect good v1.4.0 # last known good
# git checks out midpoints; test each, then:
git bisect good|bad  # repeat until it
names the culprit
git bisect reset
```

Binary search: finds the bad commit in a 1000-commit range in ~10 steps.

12 “I committed a secret / password / API key”

Rotate the secret first — assume it's compromised the moment it's pushed. Then scrub history:

```
git filter-repo --invert-paths --path .env
# or replace text everywhere:
git filter-repo --replace-text expressions.txt
```

History rewrite → **coordinate with your team, force-push, everyone re-clones. Rotating the key matters more than the scrub.**

13 “I deleted a branch / hard-reset and lost work”

```
git reflog                # every place HEAD
has been
# find the SHA from before disaster, then:
git branch rescued SHA    # or: git reset --
hard SHA
```

The reflog is local and keeps entries ~30+ days. If you committed it, it is almost never truly lost.

14 “Everything is on fire and I trust nothing”

```
cp -r project project-backup # yes, really –
snapshot first
git status && git reflog      # orient yourself
before acting
```

A filesystem copy takes 10 seconds and makes every experiment below it safe. There's no shame in the coward's backup — only in losing work.

☑ Prevention beats rescue

- Commit small and often — tiny commits are easy to undo.
- `git push` daily to a branch: the remote is your offsite backup.
- Add `.env`, keys and build dirs to `.gitignore` *before* the first commit.
- Enable `git config --global rerere.enabled true` — git remembers conflict resolutions.
- Read the command before pressing enter when it contains `-force` or `--hard`.